

Aplicação do Algoritmo de Dijkstra na Otimização de Progressões de Acordes Musicais com Inversões

Ilanna dos Reis Cardoso

Instituto Federal de Educação, Ciência e Tecnologia do Espírito Santo

Av. dos Sabiás, 330 - Morada de Laranjeiras, Serra - ES, 29166-630

cardosoilanna96@gmail.com

RESUMO

Este artigo tem como objetivo determinar o custo mínimo para executar uma progressão de acordes musicais, considerando três inversões possíveis para cada acorde. Sendo assim, aplica-se o algoritmo de Dijkstra para encontrar o menor caminho para todas as combinações possíveis, otimizando um trabalho que antes era realizado pelos acordes fundamentais sem inversão. Dessa forma, torna-se possível minimizar o esforço físico no instrumento para tocar uma determinada música, oferecendo uma solução válida para sequências que possuem mais de um acorde. Por fim, o estudo relaciona teoria de grafo e conceitos musicais para propor uma abordagem computacional que otimiza a escolha de inversões de acordes.

PALAVRAS CHAVE. Inversão, Acorde, Grafo.

1. Introdução

Ao tocar um acorde cuja tônica é a nota mais grave, diz-se que ele está no seu "estado fundamental". No entanto, é possível inverter um acorde, de modo em que sua nota mais grave não consistirá na nota fundamental, mas em outra nota que compõe o acorde determinado [Dudeque, 2003]. Assim, um acorde invertido possui as mesmas notas do acorde original, mas reorganizadas de forma que outra nota ocupe a tônica na sua formação.

Essa reorganização possui relevância prática, pois permite reduzir o deslocamento entre acordes. Por exemplo, ao tocar o acorde de C (dó maior) e, em seguida, G (sol maior), a distância entre as posições fundamentais é de sete semitons. Porém, com inversão é possível minimizar esse percurso para dois semitons, ou seja, reduz o movimento das mãos. Entretanto, em progressões maiores, não é possível afirmar com exatidão a sequência com menor custo manual. Dessa forma, este artigo tem como objetivo encontrar o menor caminho em uma progressão para todas as suas inversões possíveis.

Além de facilitar a execução, a nota mais grave é modificada e, consequentemente, altera a percepção e a expectativa do acorde para ouvintes [Schwitzgebel, 2021]. Assim, compreender e otimizar o uso de inversões torna-se relevante tanto para reduzir o movimento das mãos, quanto para construção de uma harmonia mais fluida capaz de manipular a expectativa auditiva.

O presente artigo manipula os conceitos de grafo para modelar as possíveis sequências de acordes que podem ser executados, no caso, uma progressão harmônica. Para fins de exemplificação, foi considerada uma progressão básica de C (dó maior) -> G (sol maior) -> D (ré maior). Cada acorde e suas respectivas inversões foram representados como nós do grafo, enquanto os custos de transição entre essas inversões correspondem às arestas com pesos.

Com o grafo formado, aplicou-se o algoritmo Dijkstra, com o objetivo de obter o menor caminho entre os nós do grafo proposto e, posteriormente, identificar quais inversões possuem o menor caminho. Logo, foi determinado as inversões que detém do menor custo, oferecendo maior eficiência na movimentação de acordes e maior naturalidade na execução de uma música.

2. Descrição do Problema e Referencial Teórico

Esta seção apresenta o problema central investigado neste estudo, contextualizando a problemática no cenário analisado. Em seguida, discute o referencial teórico acerca de conceitos necessários para compreender o algoritmo aplicado.

2.1. Descrição Detalhada do Problema Real

Segundo Priolli [1996] (apud Gorosito, 2021), “Dá-se o nome de acorde ao conjunto de sons ouvidos simultaneamente”. Em outras palavras, acordes são uma junção de notas executadas ao mesmo tempo. Em uma música, normalmente há progressões de acordes responsáveis por gerar uma harmonia desejada pelo compositor. Contudo, se a melodia demanda de múltiplos acordes em distâncias significativas, uma solução eficiente está no uso de inversões.

Inverter um acorde consiste em reorganizar a posição das notas do acorde [Gorosito, 2021]. Além de outros propósitos, o uso da inversão pode diminuir o deslocamento manual do instrumentista, contribuindo para uma sucessão mais suave de acordes. Na Figura 1, observa-se o acorde de C (dó maior) no seu estado fundamental e nas suas duas inversões possíveis: E (mi) como baixo na primeira inversão e o G (sol) como baixo na segunda inversão.

Figura 1: Inversões de acordes. Fonte: Gorosito (2021, p. 165), Figura 5.13.

Estado fundamental	1ª inversão	2ª inversão
Baixo - fundamental	Baixo - terça	Baixo - quinta
		

Teoricamente, a distância entre um acorde e o seguinte pode ser medida em tons e semitons. Conforme, Zamacois [2009] (apud Gorosito, 2021), "A distância mínima aceita no sistema musical é denominada meio-tom (semitom no Brasil)". Essa métrica permite quantificar o custo de deslocamento entre dois acordes distintos consecutivos, isto é, a distância (semitons) dos baixos dos acordes executados é o deslocamento de dois acordes.

Um problema clássico ao tocar uma progressão de acordes é a busca por um encadeamento suave entre os acordes em uma música, chamado de *voice leading*. Em alguns cenários, a distância direta entre dois acordes no estado fundamental pode gerar deslocamentos longos e pouco naturais. Sendo assim, um músico que entende de inversões, analisa quais são suas possibilidades para naturalizar e diminuir o percurso ao tocar uma música. No entanto, em músicas com múltiplos acordes, é comum que o instrumentista possua maior dificuldade em encontrar um caminho com o menor peso possível.

Para formalizar esse problema de maneira matemática, cada acorde e suas respectivas inversões são representados como nós de um grafo, enquanto as transições possíveis entre eles formam arestas ponderadas. O custo de cada aresta corresponde ao número de semitons percorridos pelo baixo de uma inversão até o acorde subsequente. Assim, o problema de encontrar a progressão mais eficiente é determinar o menor custo total entre o primeiro e o último acorde da sequência, tarefa em que o algoritmo de Dijkstra será executado para encontrar o caminho mínimo.

2.2. Referencial Teórico

O algoritmo que será implementado no estudo é chamado de Algoritmo de Dijkstra, também conhecido como algoritmo de caminho mínimo [Gersting, 2004]. O algoritmo de Dijkstra é utilizado para encontrar o caminho mais curto entre dois vértices em um grafo ponderado. Dessa forma, o algoritmo percorre todos os nós do grafo, usando o princípio de relaxamento de arestas, atualizando com a menor distância caso encontre um custo menor e, posteriormente, modificando o predecessor [Rodrigues, 2023]. Além disso, o algoritmo determina o nó analisado pelo vértice com a menor distância no momento atual. Em relação a complexidade, Dijkstra no laço *para* utiliza de $O(n)$ operações [Gersting, 2004]. Para encontrar o caminho mínimo entre x e y , no pior caso, o último nó a ser incluído na lista dos inspecionados, é y . Por isso, o laço *enquanto* é executado $n - 1$ vezes [Gersting, 2004]. Ou seja, no pior caso, a complexidade do algoritmo proposto é de $O(n^2)$, devido as operações no laço, inicialização e escrita de resultados [Gersting, 2004].

No problema em questão, é necessário minimizar percurso físico para tocar uma progressão musical. Ao representar essa situação como um grafo, sem arestas com pesos negativos, a solução corresponde ao caminho mínimo entre nós. Portanto, o algoritmo de Dijkstra é o mais adequado para a resolver o problema, pois seu objetivo principal é identificar o caminho mínimo entre dois vértices de um grafo [Rodrigues, 2023], o que se alinha diretamente com demanda de encontrar o melhor percurso na execução de uma música.

Existem alguns estudos nacionais escritos para relacionar grafos e teoria musical. Como por exemplo, o artigo: *Análises Melódicas usando Grafos*, redigido por Furtado e Longo [2024] pelo Instituto de Informática da UFG. O artigo propõe uma abordagem de percorrer um grafo de uma sucessão musical, usando busca em nível, gerando uma harmonia qualquer, e assim, analisar as melodias encontradas. Além disso, Queiroz [2010] no artigo *Uma Notação Musical para Representação de Progressões Harmônicas Utilizando Grafos* utiliza grafos direcionados para criar uma notação musical chamada de Diagrama de Estados Musicais Harmônicos (DEMH). De maneira semelhante, a dissertação de mestrado de Athayde [2020], intitulada de *Teoria dos grafos aplicada à análise musical*, sugere o uso de grafo para representar estruturas musicais complexas para análise musical. Por último, o trabalho de conclusão de curso da UTFPR de Wieler [2023], chamado de *Análise Sintática e Semântica Musical Usando Grafos* usa grafos para representações musicais na qual as notas são vértices e as arestas representam a progressão melódica, recebendo pesos através de um Sistema de Intervalos Generalizado (GIS). Esse artigo implementa o algoritmo VF2, baseado em busca em profundidade, para identificar isomorfismos entre os grafos, para assim, constituir uma análise semântica da melodia.

Esses trabalhos apresentam abordagens semelhantes a este estudo. No entanto, nenhum deles contempla a **otimização de progressões de acordes com inversões utilizando algoritmos de caminho mínimo**, lacuna que este estudo pretende preencher.

3. Modelagem

A modelagem se faz necessária para compreender o papel de cada ator no problema. Dessa forma, o grafo representa uma progressão de acordes base para exemplificar o problema. Nesse caso, a sequência escolhida foi: C (dó maior) -> G (sol maior) -> D (ré maior). Cada acorde possui duas inversões e o acorde fundamental. Essas são as inversões possíveis para cada acorde:

Tabela 1: Inversões dos acordes da progressão. Fonte: Elaborado pelo Autor

Acorde	Estado Fundamental	1ª Inv.	2ª Inv.
C (Dó)	Dó-Mi-Sol	Mi-Sol-Dó	Sol-Dó-Mi
G (Sol)	Sol-Si-Ré	Si-Ré-Sol	Ré-Sol-Si
D (Ré)	Ré-Fá#-Lá	Fá#-Lá-Ré	Lá-Ré-Fá#

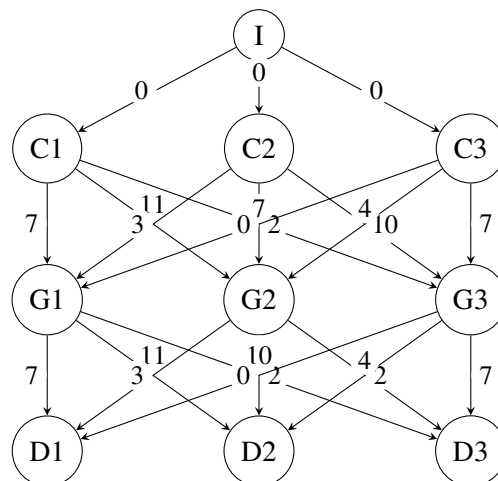
No contexto do problema, cada acorde, assim como suas inversões, é representado como um vértice (nó) no grafo. Isso significa que cada aresta (arco) representa o percurso das mãos do músico no instrumento, permitindo que ele percorra o espaço necessário de um acorde até o seguinte na progressão musical. A medida da distância são os pesos, que correspondem aos semitons necessários para alcançar o próximo acorde. Dessa forma, o grafo é modelado de maneira que garante ao algoritmo percorrer todas as possibilidades e determinar o caminho mais curto entre os elementos da musica. Abaixo, estão escritos os pesos do grafo em semitons:

Tabela 2: Custos de transição entre acordes em semitons. Fonte: Elaborado pelo Autor

Arestas	C → G			G → D		
Nó origem	C1	C1	C1	G1	G1	G1
Nó destino	G1	G2	G3	D1	D2	D3
Peso	7	11	2	7	11	2
Nó origem	C2	C2	C2	G2	G2	G2
Nó destino	G1	G2	G3	D1	D2	D3
Peso	3	7	10	3	10	2
Nó origem	C3	C3	C3	G3	G3	G3
Nó destino	G1	G2	G3	D1	D2	D3
Peso	0	4	7	0	4	7

Vale ressaltar, a criação de um nó articial *I* para encontrar o menor caminho de maneira global. Dessa forma, o nó *I* possui peso nulo para identificar, a partir de C1, C2 ou C3, o percurso mais econômico. A seguir, apresentam-se a representação do grafo ponderado correspondente:

Figura 2: Grafo direcionado com pesos entre níveis C, G e D. Fonte: Elaborado pelo Autor



Para descobrir o menor esforço físico entre os acordes, será utilizado o algoritmo para o

caminho mínimo, chamado de algoritmo de Dijkstra. Esse algoritmo tem a função de encontrar a distância mínima entre dois nós que possuem o custo **não negativo** [Gersting, 2004], no caso em questão, entre os acordes. Abaixo, está o algoritmo de Dijkstra:

Algorithm 1 Algoritmo de Dijkstra. Fonte: Resendo (Slides - 6.3)

1. Ler $G = (N, A)$, c_{ij} (não negativo) é a distância entre os nós e nó de origem s .
 2. **Iniciar variáveis:** $S := \emptyset$, $R := N$.
 3. $d(i) := \infty \forall i \in N$, $d(s) := 0$, $pred(s) := 0$.
 4. **Enquanto** $|S| < n$ **fazer**
 5. Seja $i \in N$ tal que $d(i) = \min\{d(j) : j \in R\}$;
 6. $S := S \cup \{i\}$, $R := R - \{i\}$;
 7. **Para** $j \in N$ com $(i, j) \in A$ **fazer**
 8. **Se** $d(j) > d(i) + c_{ij}$ **então** $d(j) := d(i) + c_{ij}$;
 9. $pred(j) := i$;
 10. **Fim enquanto**
-

O algoritmo inicia com dois conjuntos principais: S , que armazena os nós já visitados e R para nós não visitados. O usuário seleciona a origem, que corresponde ao nó inicial do percurso. Além disso, inicia-se cada elemento do vetor d , que guarda a distância mínima da origem s até cada nó, com o valor infinito para todos os nós, exceto para a origem, que recebe $d(s) = 0$. O vetor $pred$, vetor que registra os predecessores de cada nó no caminho mínimo, é inicializado com $pred(s) = 0$, considerando que da origem até a ela mesma não possui predecessor.

O processamento principal, ocorre dentro de um laço que continua enquanto todos os nós não tiverem sido visitados ($|S| < n$). A cada iteração, seleciona-se o nó i em R com a menor distância ($d(i) = \min\{d(j) : j \in R\}$). Esse nó, então, é movido de R para S , indicando que já foi um nó analisado.

Posteriormente, para cada nó j adjacente ao nó i , verifica-se se o $d(j) > d(i) + c_{ij}$, ou seja, se o caminho passando por i oferece uma distância menor do que o caminho em $d(j)$. Se sim, altera $d(j)$ com essa nova distância e $pred(j)$ com o nó i , garantindo que mantenha o caminho de menor custo da origem até qualquer nó. Dessa forma, ao finalizar os dois loops, encontra-se o menor custo da origem até qualquer nó e o predecessor para alcançar o nó final.

Para armazenar o grafo digitalmente e poder manipula-lo, é necessário representar de maneira computacional. Normalmente, as representações utilizam matriz de adjacência ou uma lista de adjacências [Gersting, 2004]. Abaixo, estão as duas formas de representar um grafo no computador:

Tabela 3: Matriz de adjacências. Fonte:
elaborada pelo autor.

	<i>I</i>	<i>C1</i>	<i>C2</i>	<i>C3</i>	<i>G1</i>	<i>G2</i>	<i>G3</i>	<i>D1</i>	<i>D2</i>	<i>D3</i>
<i>I</i>	0	1	1	1	0	0	0	0	0	0
<i>C1</i>	0	0	0	0	1	1	1	0	0	0
<i>C2</i>	0	0	0	0	1	1	1	0	0	0
<i>C3</i>	0	0	0	0	1	1	1	0	0	0
<i>G1</i>	0	0	0	0	0	0	0	1	1	1
<i>G2</i>	0	0	0	0	0	0	0	1	1	1
<i>G3</i>	0	0	0	0	0	0	0	1	1	1
<i>D1</i>	0	0	0	0	0	0	0	0	0	0
<i>D2</i>	0	0	0	0	0	0	0	0	0	0
<i>D3</i>	0	0	0	0	0	0	0	0	0	0

Tabela 4: Lista de adjacências. Fonte:
elaborada pelo autor.

Nó	Liga para
<i>I</i>	<i>C1</i> , <i>C2</i> , <i>C3</i>
<i>C1</i>	<i>G1</i> , <i>G2</i> , <i>G3</i>
<i>C2</i>	<i>G1</i> , <i>G2</i> , <i>G3</i>
<i>C3</i>	<i>G1</i> , <i>G2</i> , <i>G3</i>
<i>G1</i>	<i>D1</i> , <i>D2</i> , <i>D3</i>
<i>G2</i>	<i>D1</i> , <i>D2</i> , <i>D3</i>
<i>G3</i>	<i>D1</i> , <i>D2</i> , <i>D3</i>
<i>D1</i>	–
<i>D2</i>	–
<i>D3</i>	–

4. Resultados

Esta seção têm como objetivo apresentar a aplicação do algoritmo escolhido e resolvendo o problema central do artigo. Além da implementação, a seção também demonstra os resultados obtidos com a pesquisa e a implementação do algoritmo. Assim, é possível elucidar as apurações feitas ao longo desse estudo.

4.1. Aplicação Passo a Passo do Algoritmo

Tabela 5: Tabela inicial de valores de $d(i)$ e $pred(i)$

	$d(i)$	$pred(i)$
<i>I</i>	∞ 0	0
<i>C1</i>	∞ 0	<i>I</i>
<i>C2</i>	∞ 0	<i>I</i>
<i>C3</i>	∞ 0	<i>I</i>
<i>G1</i>	∞ 7 3 0	<i>C1</i> <i>C2</i> <i>C3</i>
<i>G2</i>	∞ 11 7 4	<i>C1</i> <i>C2</i> <i>C3</i>
<i>G3</i>	∞ 2	<i>C1</i>
<i>D1</i>	∞ 7 0	<i>G1</i> <i>G3</i>
<i>D2</i>	∞ 11 4	<i>G1</i> <i>G3</i>
<i>D3</i>	∞ 2	<i>G1</i>

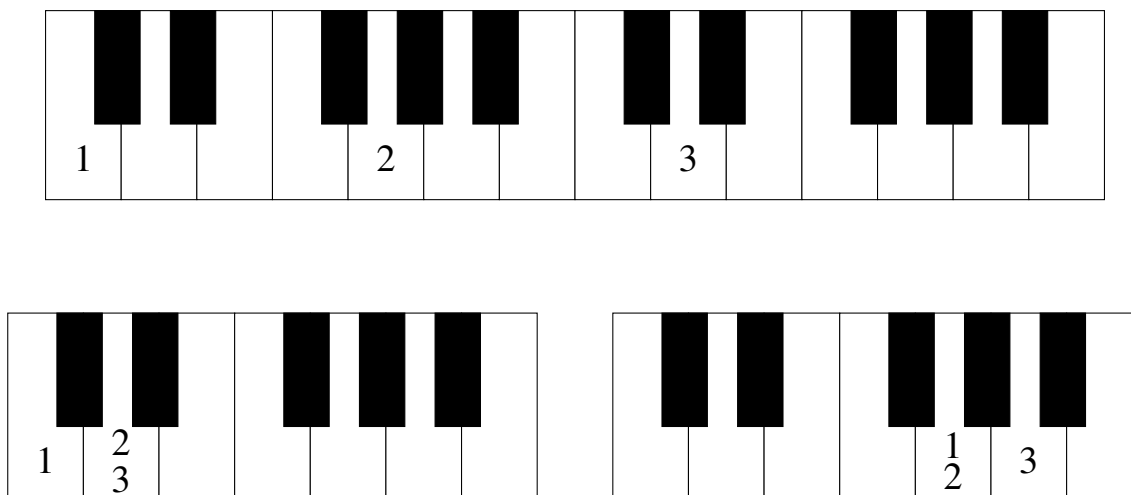
Tabela 6: Passos Otimizados do Algoritmo de Dijkstra. Fonte: Elaborado pelo autor

Iteração	Nó i e Conjuntos S, R	Atualizações de Distância $d(i)$ e Predecessor $pred(i)$
Iniciais	$S = \emptyset, R = N$	$d(I) = 0, pred(I) = 0, d(i) = \infty$.
1	$i = I, j = \{C1, C2, C3\}$	$d(C1) = 0, pred(C1) = I; \quad d(C2) = 0, pred(C2) = I; \quad d(C3) = 0, pred(C3) = I$.
2	$i = C1, j = \{G1, G2, G3\}$	$d(G1) = 7, pred(G1) = C1; \quad d(G2) = 11, pred(G2) = C1; \quad d(G3) = 2, pred(G3) = C1$.
3	$i = C2, j = \{G1, G2, G3\}$	$d(G1) = 3, pred(G1) = C2; \quad d(G2) = 7, pred(G2) = C2; \quad d(G3) : \text{Falso}$.
4	$i = C3, j = \{G1, G2, G3\}$	$d(G1) = 0, pred(G1) = C3; \quad d(G2) = 4, pred(G2) = C3; \quad d(G3) : \text{Falso}$.
5	$i = G1, j = \{D1, D2, D3\}$	$d(D1) = 7, pred(D1) = G1; \quad d(D2) = 11, pred(D2) = G1; \quad d(D3) = 2, pred(D3) = G1$.
6	$i = G3, j = \{D1, D2, D3\}$	$d(D1) = 2, pred(D1) = G3; \quad d(D2) = 6, pred(D2) = G3; \quad d(D3) : \text{Falso}$.
7	$i = D1, j = \emptyset$	Não há atualizações.
8	$i = D3, j = \emptyset$	Não há atualizações.
9	$i = G2, j = \{D1, D2, D3\}$	$d(D1) : \text{Falso}; \quad d(D2) : \text{Falso}; \quad d(D3) : \text{Falso}$.
10 (Fim)	$i = D2, j = \emptyset$	Todos os nós visitados.

4.2. Resultados Obtidos

Iniciando em I , existem 27 caminhos diferentes, com custos diferentes. Nesse sentido, os caminhos que oferecem os menores custos são: $C1 \rightarrow G3 \rightarrow D1$ e $C3 \rightarrow G1 \rightarrow D3$. Esses caminhos geram um custo de dois semitons, sendo um percurso bem mais curto que o original de 14 semitons. Dessa forma, com o algoritmo proposto em conjunto com os conceitos de teoria dos grafos, foi possível otimizar a sequência e encontrar alternativas mais eficientes. No primeiro teclado na figura abaixo, evidencia-se a sucessão das tônicas dos acordes no estado fundamental ($C1 \rightarrow G1 \rightarrow D1$). A partir do algoritmo proposto, o segundo e o terceiro teclado ilustram os caminhos otimizados, correspondendo a $C1 \rightarrow G3 \rightarrow D1$ e $C3 \rightarrow G1 \rightarrow D3$.

Figura 3: Percursos entre tônicas da progressão, mostrando diferentes caminhos mínimos. Fonte: Elaborado pelo autor



5. Conclusão

Ao longo deste artigo, foi estudado tanto os conceitos de grafo, quanto de música para relacionar de forma apropriada a problemática principal. Foram aplicados grafo e algoritmo que contabiliza o caminho mínimo dos acordes possíveis em uma progressão. Assim, o problema foi resolvido com algoritmo de Dijkstra, sendo válido para encontrar o percurso mínimo.

O trabalho contribuiu para o aprendizado da disciplina, integrando teoria e prática, e reforçou conhecimentos musicais previamente adquiridos.

Entre as limitações, destacam-se as dúvidas durante a disciplina e a simplificação da modelagem, como a consideração de apenas uma progressão fixa e a medição do custo apenas em semitons. Futuras melhorias podem incluir especificações de cada instrumento, considerando suas especificidades.

Por fim, o artigo propôs um problema para mitigar o custo físico entre acordes. De forma simplificada, o problema obteve uma resolução com teoria de grafos e o algoritmo apropriado para encontrar o caminho mínimo.

Referências

- ATHAYDE, E. M. **Teoria dos grafos aplicada à análise musical**. Belo Horizonte, MG, 2020.
- CROSATO, L. D. C. **Caracterização musical via espectro de grafos**. Curitiba: Universidade Tecnológica Federal do Paraná, 2022.
- DUDEQUE, N. **Harmonia Tonal**. 1. ed. 2003.
- FURTADO, V.; LONGO, B. **Análises Melódicas usando Grafos**. Goiânia: Instituto de Informática, Universidade Federal de Goiás, 2024.
- GERSTING, J. **Fundamentos Matemáticos para a Ciência da Computação**. 1. ed. São Paulo, 2004.
- GOROSITO, L. **Notação e Linguagem Musical**. 1. ed. Curitiba: Intersaberes, 2020. E-book. Disponível em: <https://plataforma.bvirtual.com.br>. Acesso em: 27 nov. 2025.
- OPUS 3 | APRENDA MÚSICA. **VOICE LEADING: ELEVE seu Acompanhamento para um OUTRO PATAMAR**. YouTube, 2023. Disponível em: https://youtu.be/Y_Q95PXQqWE?si=DZUEFYxplHIVuhUm. Acesso em: [20 nov. 2025].
- PRIOLLI, M. L. M. **Princípios básicos da música para a juventude**. Rio de Janeiro: Editora Musimed, 1996.
- QUEIROZ, A. A. **Uma notação musical para representação de progressões harmônicas utilizando grafos**. Campinas: UNICAMP, 2010.
- RESENDÓ, L. **Slides - 6.3**. Slides da disciplina, 2024.
- RODRIGUES, G. C. da S. **Análise de desempenho do algoritmo de Dijkstra com diferentes estruturas de dados**. Universidade Federal do Espírito Santo, Alegre – ES, 2023.
- SCHWITZGEBEL, K. **Effects of Chord Inversion and Bass Patterns on Harmonic Expectancy in Musicians**. Received: August 19, 2020, accepted May 4, 2021.
- WIELER, P. **Representações musicais usando grafos para aperfeiçoamento da análise musical**. [Local]: [Instituição], 2023.
- ZAMACOIS, J. **Tratado de armonía**. Barcelona: Labor, 2009.